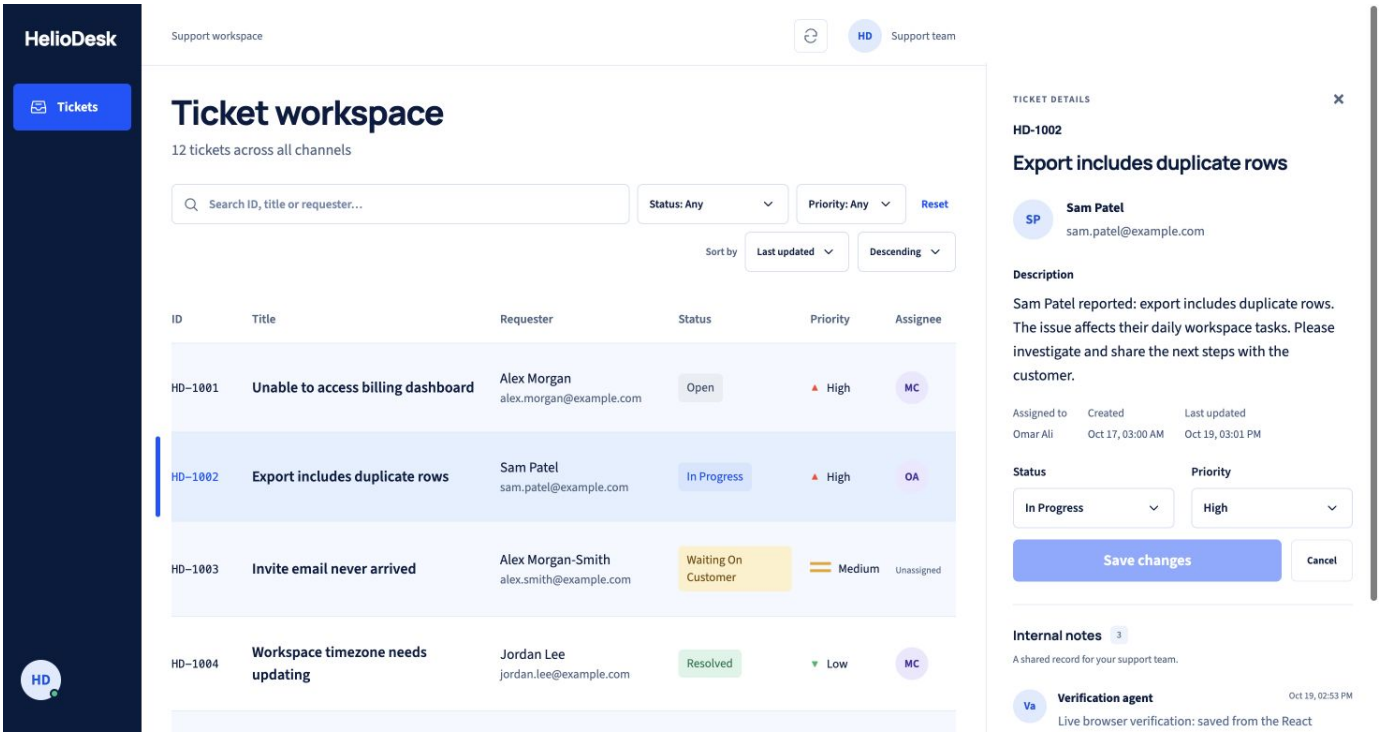
HelioDesk

TENX / FULL-STACK ASSESSMENT

A clear workspace. A verified system.

An illustrated tour of the requirements, implementation and evidence behind a Flask + React ticket platform.



Live app capture: ticket queue and a visible detail inspector. Screenshots show the running product, not the generated design concepts.

12
seeded tickets

30
automated tests passed

01
selected E2E bonus

Open the live app

Explore the public repository

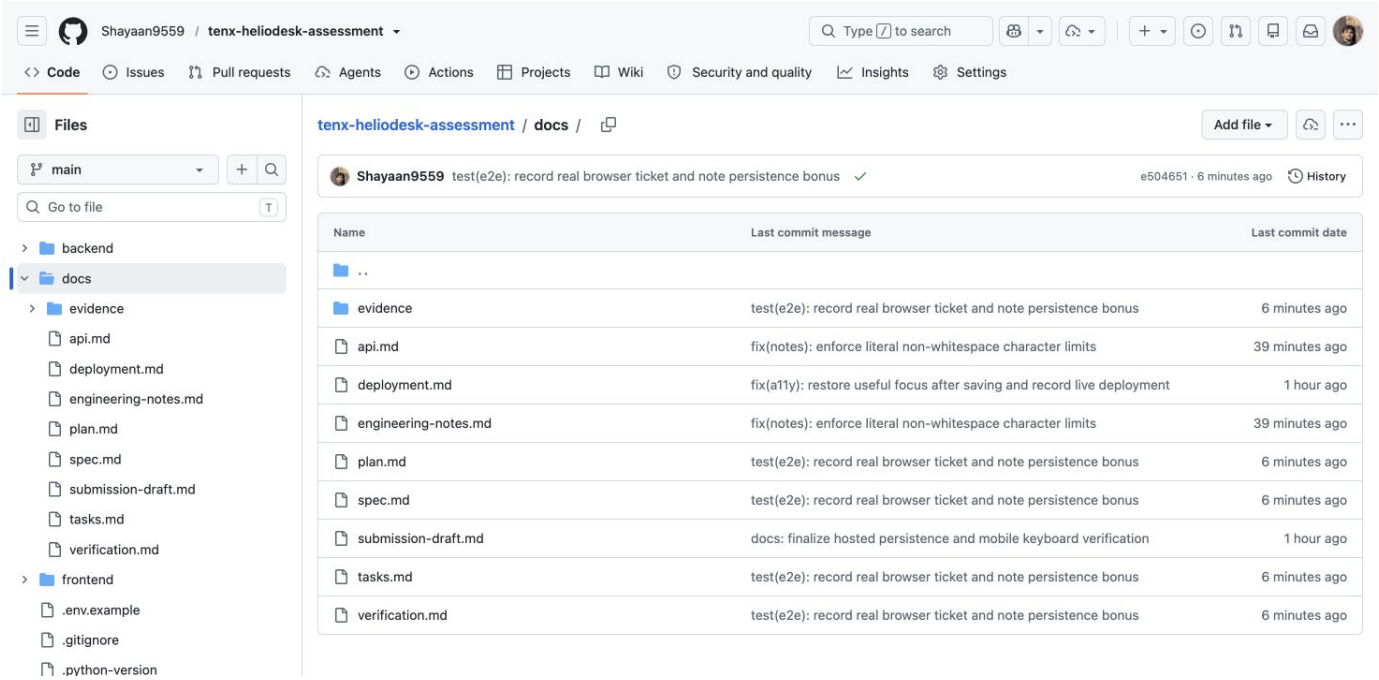
Prepared for review by Shayaan Shahid. This companion summarizes the work; the repository contains the full specification, source, tests and verification record.

01 / SPECIFY, CONTRACT, PLAN

From requirement to evidence

The assignment asks for a traceable engineering process. Each document has a specific job, and stable IDs connect the requirements to tasks and checks.

REPOSITORY ARTIFACT	PURPOSE
docs/spec.md	What must work: FR, NFR and AC identifiers.
docs/api.md	How React and Flask communicate: payloads, status codes and errors.
docs/plan.md	Architecture, schema, boundaries, risks and testing approach.
docs/tasks.md	Ordered implementation tasks linked to requirements.
docs/verification.md	Pass results, manual observations, evidence and limitations.



Actual GitHub docs directory. Reviewable commits begin with the specification and API plan, then backend, frontend, verification and focused fixes.

EXAMPLE: A SAVED TICKET

FR-05 / AC-05 requires persistent edits. API-05 defines PATCH. service.update_ticket() validates and commits. Backend, React and browser E2E checks verify the saved response and persistence after refresh.

02 / BUILD WITH CLEAR RESPONSIBILITIES

How the pieces fit together

React handles the screen. A focused API client sends HTTP requests. Flask validates and runs the query or write. The relational database stores the result.

REACT	HTTP CLIENT	FLASK	DATABASE
App + TicketDialog	frontend/src/api.ts	routes -> service	Ticket -> Note

One deployment, two runtimes

Vercel serves the built React assets and runs Flask for API requests. Flask alone connects to Supabase Postgres. Local development and assessment tests use SQLite; the browser never talks directly to a database or imports seed data.

backend/app/routes.py | actual excerpt

```
@api.patch('/tickets/<ticket_id>')
def patch(ticket_id):
    return jsonify(service.update_ticket(
        ticket_id, request.get_json(silent=True)))
```

The route stays thin: HTTP comes in, the service owns validation and the write, and the saved ticket comes back as JSON.

backend/app/service.py | actual excerpt

```
for field, value in payload.items():
    setattr(ticket, field, value)
advance_timestamp(ticket)
db.session.commit()
return ticket.serialize(detail=True)
```

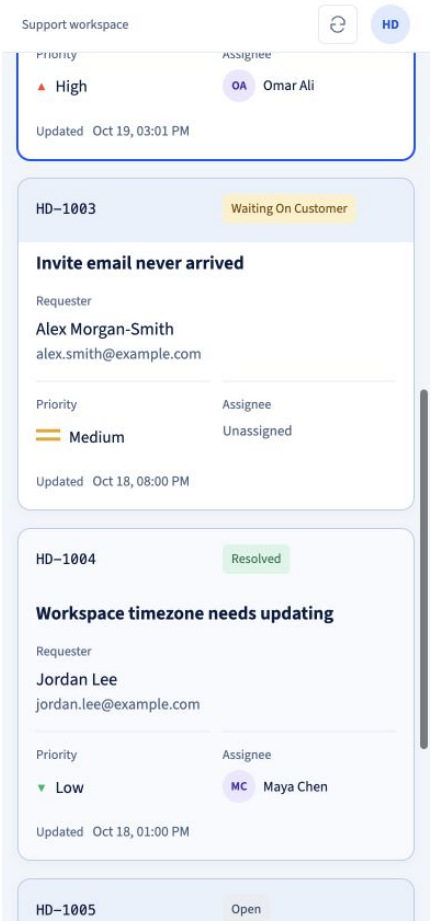
Validation occurs before this block. Ticket/note changes and updated_at commit together; failed writes are rolled back by the error handler. Only a committed response becomes React's saved state.

Repository map
backend/app: models, routes, service, seed and migration
frontend/src: API client, workspace, inspector and custom CSS
backend/tests + frontend/src/App.test.tsx + frontend/e2e: three test layers

03 / DELIVER THE REQUIRED WORKFLOW

Usable on desktop and mobile

The selected navy-and-cobalt visual system uses clear typography, text status labels, consistent controls and distinct mobile cards. Clicking a ticket row opens its details.



A complete vertical slice

Search, status/priority filters, sort order and five-item pagination combine on the server. Query changes reset to page 1; opening details preserves the current list state.

JSON REST ENDPOINT	BEHAVIOR
GET /api/health	Service health
GET /api/tickets	Query + pagination
GET /api/tickets/:id	Details + notes
PATCH /api/tickets/:id	Status / priority
POST /api/tickets/:id/notes	Append a note

Readable mobile cards

Visible blue-gray borders, tinted ID/status headers, white surfaces and 16px gaps distinguish tickets. At 320px, recorded checks found no page-level horizontal scrolling.

Keyboard and asynchronous states

Visible focus, Escape/return focus, native modal containment at smaller widths, loading, saving, success and field errors. Abort + request generation prevent an older response replacing the newest search.

Priority: Urgent > High > Medium > Low. Resolved tickets can reopen. Dates are ISO 8601 UTC in the API and localized in the browser.

Rule 4, followed literally: notes require 3-500 non-whitespace characters; interior spaces/newlines remain readable. Author is required (1-80 trimmed characters). Flask enforces these rules even if React is bypassed.

04 / VERIFY, THEN SHOW THE EVIDENCE

30 tests. Three useful layers.

Fresh verification on 19 October 2026, source e504651. These are captured runner outputs, reflowed in a local evidence page and then screenshotted for readability.

HELIO DESK / VERIFICATION

30 automated tests passed

19 October 2026 | Captured command output, reflowed for readability.
Real runner output; not an application screenshot or simulated result.

Flask / pytest — 17 passed

```
..................
[100%]
17 passed in 0.21s
```

React / Vitest — 12 passed

```
Test Files  1 passed (1)
      Tests  12 passed (12)
   Start at  16:49:53
   Duration  1.47s (transform 65ms, setup 124ms, import 86ms, tests 846ms,
environment 340ms)
```

Real browser / Playwright — 1 passed

```
Running 1 test using 1 worker
  ✓  1 [chromium] › e2e/ticket-persistence.spec.ts:3:1 › AC-04/05/07: real
browser saves ticket and note, then reloads persisted data (1.3s)
  1 passed (2.5s)
```

17 backend: isolated SQLite, queries, writes, validation, 404s, rollback and migration.
12 frontend: combined controls, UI states, saved responses, notes, focus and stale requests.
1 E2E: real Chromium + HTTP + Flask + SQLite; persistence after page reload.

Run from repository root after the README setup:

```
.venv/bin/python -m pytest backend/tests -q
npm test --prefix frontend                    npm run test:e2e --prefix frontend
```

05 / COVERAGE, BONUS AND HANDOVER

What was checked - and what was not

The acceptance matrix records evidence rather than a predicted score. Detailed methods, screenshots and known limits remain in docs/verification.md.

CRITERIA	RESULT	RECORDED EVIDENCE
AC-01-03	Pass	Five newest results, totals, combined queries, sorting and pagination.
AC-04-05	Pass	Canonical details, preserved list state, saved edits survive refresh.
AC-06-08	Pass	Field errors; failed writes unchanged; one saved note; length limits.
AC-09-11	Pass	Missing ticket recovery, manual Retry recovery, stale response guard.
AC-12	Pass	Chrome keyboard workflows and 320px layout; full audit not claimed.

Selected bonus: automated E2E

The browser edits HD-1001, adds a note, reloads and checks the saved values/history. An independent GET confirms exactly one matching note. A fresh temporary SQLite database keeps hosted and development data untouched.

frontend/e2e/ticket-persistence.spec.ts | actual excerpt

```
const response = await request.get('/api/tickets/HD-1001');
expect(response.status()).toBe(200);
const saved = await response.json();
expect(saved.status).toBe('in_progress');
expect(saved.priority).toBe('urgent');
```

Watch the committed E2E recording

Trade-offs and limits. One small monolith, server-confirmed saves and no authentication (per brief). SQLite is reproducible locally; Postgres provides durable hosted data. The E2E test adds a browser download/startup cost and covers one Chromium desktop path. Safari/Firefox, full screen-reader certification and production load/security audits are not claimed. Provider cold starts remain possible.

AI disclosure. OpenAI Codex assisted specification, code, tests, design and debugging. The README identifies corrected output and verification. The candidate remains responsible for understanding the work. Submit repo/app links, actual time spent and a concise trade-off; this PDF is optional supporting evidence.